

SOLID/GRASP Principles

Single Responsibility Principle:

The design that was implemented follows the single responsibility principle because it ensures that each class has only one clear purpose in its functionality. Some examples include:

- The *Project* class only manages project level data such as the name, description, and lists of tasks and members
- The *Task* class handles things that are related to tasks, such as the title, status, and priority of the task itself
- The *TeamMember* represents a class that is only responsible for information about individual members and their assigned projects.

This separate of classes helps to keep the system modular and easy to test.

Open/Closed Principle:

This SOLID principle stands for open to extension but closed for modification, done by the usage of inheritance and abstraction. The base class *Task* defines the shared fields and behavior, while there are subclasses of this *Task* class such as *RepeatingTask* or *HighPriorityTask* that extend upon the original class to add specialized functionality. By doing this, new task types can be added without having to touch any existing code, which reduces the risk of regression and helps to maintain clean stable behavior across the entire project as a whole.

Dependency Inversion Principle:

The code applies the dependency inversion principle since it depends on the interfaces themselves instead of concrete classes. The high-level components in the code interact with the *MainTask* interface instead of a specific task implementation. The structuring of this principle helps to substitute different task types and perform unit testing with mocks, and integrate future extensions without having to recode the project logic.

Information Expert Principle:

The information expert principle states that classes should only be assigned responsibilities if they have the necessary information to complete those responsibilities. The *Project* class manages adding and removing tasks while owns the task list, and each *Task* class updates its own status and due date. Simply, the *Project* class has responsibilities of tasks since it has the information of different tasks, and the same applied to *Task*, where contains its own information about a specific task. By forming this kind of behavior with the class that has relevant information, this system can avoid redundancy and and remains clear.

Low coupling and High Cohesion:

The code minimizes dependencies between classes while keeping related behavior grouped together, which is what the low coupling and high cohesion principle states. Each class performs

their own set of focused actions relevant to its purpose, which leads to high cohesion. Classes like *Project* don't handle the internal logic of *Task*, and *TeamMember* only maintains its own contact and role data. This low coupling, high cohesion protocol helps a lot in keeping components reusable, maintainable, and adaptable.

Participating Members:

Austin, Yitian, Shreya, Nehal, Joseph

Link to document:

https://docs.google.com/document/d/16Xj8_WulRaVg-OKjobaMJwEHESh4AjdY_Ducy1EppHE/edit?usp=sharing